Pix2Pix GAN for Image-to-Image Translation

Joyce Henry, Terry Natalie, Den Madsen

I. Introduction

The Pix2Pix Generative Adversarial Network, or GAN, is an approach to training a deep convolutional neural network for image-to-image translation tasks. The careful configuration of architecture as a type of image-conditional GAN allows for both the generation of large images compared to prior GAN models (e.g. such as 256×256 pixels) and the capability of performing well on a variety of different image-to-image translation tasks. In this tutorial [1], you will discover how to develop a Pix2Pix generative adversarial network for image-to-image translation. Pix2Pix is a Generative Adversarial Network, or GAN, model designed for general purpose image-to-image translation.

The GAN architecture is comprised of a generator model for outputting new plausible synthetic images, and a discriminator model that classifies images as real (from the dataset) or fake (generated). The discriminator model is updated directly, whereas the generator model is updated via the discriminator model. As such, the two models are trained simultaneously in an adversarial process where the generator seeks to better fool the discriminator and the discriminator seeks to better identify the counterfeit images. The Pix2Pix model is a type of conditional GAN, or cGAN, where the generation of the output image is conditional on an input, in this case, a source image. The discriminator is provided both with a source image and the target image and must determine whether the target is a plausible transformation of the source image. The generator is trained via adversarial loss, which encourages the generator to generate plausible images in the target domain. The generator is also updated via L1 loss measured between the generated image and the expected output image. This additional loss encourages the generator model to create plausible translations of the source image. The Pix2Pix GAN has been demonstrated on a range of image-to-image translation tasks such as converting maps to satellite photographs, black and white photographs to color, and sketches of products to product photographs. Now that we are familiar with the Pix2Pix GAN, let's prepare a dataset that we can use with image-to-image translation.

II. Related Work

Image synthesis and image translation has always been a significant research field in computer vision. Many existing methods, such as autoregressive model, deterministic network, and variational autoencoders, have achieved excellent results. Compared with these methods, the generative adversarial networks have shown better performance in image generation. In the beginning, GAN uses noise vectors of Gaussian or uniform distribution to synthesize images. The problem with this method is that it is impossible to de-termine the type of synthetic image. Therefore, GAN introduces conditional variables based on GAN, which can use category labels or related attributes to determine the image type. On the one hand, this method achieves pretty results; on the other hand, it can determine the type of synthetic image artificially. Al-though CGAN realizes the control of image type, it is powerless for the specific content of the synthetic image. To solve this problem, Reed et al. proposed image synthesis based on the text description. The text description contains the basic object category information, and it can determine the specific content of the image. Therefore, text to image synthesis has a high degree of flexibility.

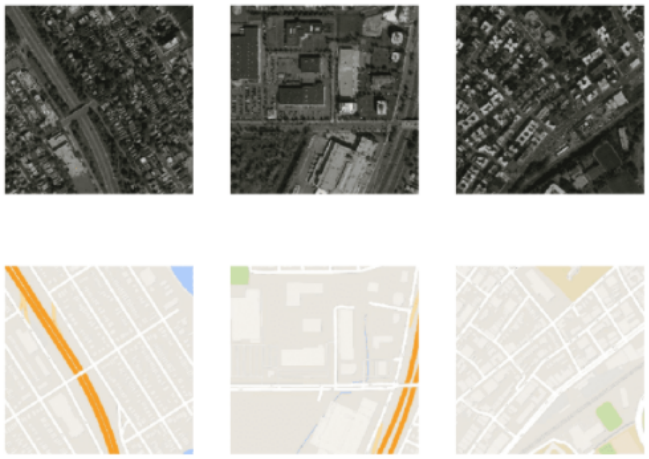


Fig. 1: Plot of Three Image Pairs Showing Satellite Images (top) and Google Map Images (bottom).

In this section, we first list some well-known works that have benefited from GAN, and then we will focus on various applications, including medical imaging and image-to-image conversion. Literature review shows that few review papers on GAN architecture and performance are available. The generative model [1], [2], [3] assumes that the data is created by a specific distribution defined by two parameters (ie Gaussian distribution) or a non-parametric variant (each instance has its own contribution to the distribution), and it uses Specific algorithms are used to approximate the underlying distribution. This method enables the generative model to generate data, not just distinguish data (classification) [4] [5] [6] and generate invisible data. In this article, we collected a wide range of GAN models and discussed them in detail. In order to avoid being interrupted in our elaboration, we first show the original GAN definition, and then explain its changes in the following subsections.

Another method combines cVAE-GAN [7], [8], [9] and

[1] https://machinelearningmastery.com/how-to-develop-a-pix2pix-gan-for-image-to-image-translation/

cLR-GAN model [10], [11], [12] to generate diverse and realistic outputs. In detail, they utilize cVAE-GAN [7], [8], [9] to encode ground truth target image x_B into a latent space Z and the generator uses input source image x_A conditioned with randomly sampled latent code z to produce translated image x_{AB}. The process can be denoted as $x_B \mapsto z \mapsto x_{AB}$. Then, they exploit cLR-GAN model [10], [11], [12] which generates translated output image x_{AB} with input x_A and random latent code z and then tries to reconstruct the latent code from x_{AB}. Similarly, the training process can be denoted as $z \mapsto x_B \mapsto z$. By combining the two objectives into a hybrid model, BicycleGAN can generate diverse and realistic outputs. They train a CNN-based regressor to regress an overly smoothed image from an incomplete input. Then, they performed a pixelwise nearest-neighbor queries to match pixels using multi-scale descriptors and generated multiple high-quality, high-frequency outputs in a controllable manner. In detail, they first employ cross-domain autoencoders to get disentanglement components of source and target images, where exclusive representations are learned by Gradient Reversal Layer (GRL) and shared representations are learned by the $\mathcal{L}_1$ loss with adding noise on features of two domains. The multi-modal translated results are obtained by using random noise as exclusive representation.

In progressive GANs, the model expands the architecture of the standard network [13] where the idea was extracted from progressive neural networks. This model has high performance as it can receive additional leverage via lateral connections to earlier learned features. This architecture is widely used for extracting complex features. For training, the model starts with low resolution images and progressively G and D grow to reach the desirable results. It is worth of mentioning, during this growing process, all the variables remain trainable. Taigman et al. [14] present a domain transfer network (DTN) for unpaired cross-domain image generation by assuming constant latent space between two domains, which could generate images of the target domains' style and preserve their identity. Similar to the idea of dual learning in neural machine translation, DualGAN [15], This progressive training strategy helps the networks to be of stable learning. Currently, several state-of-the-art GANs adopted such training strategy to improve their overall performance. Heljakka et al. adopted the progressive GANs into Autoencoder network for image reconstruction. The authors claim this model has promising results in image synthesis and inference. However, the model is only evaluated in CelebA dataset and the efficiency of the proposed model is not evaluated. map a pair of source image and target image into one same latent code in a shared latent space. They used a VAE-GAN network to model each image domain and achieved cross-domain translation by a cycle-consistency constraint and weight-sharing constraint. Their model can effectively handle the translation with holistic changes or large shape changes. They first sketch a primary coarse result in low-resolution adopted a similar architecture of CycleGAN [16]. And then they deployed a stacked structure to refine result with more details and high resolution by multiple refinement processes.

From the above basic definition, we see that converting an image from one source domain to another target domain can cover many problems in computer vision, computer graphics. Specifically, GAN models has been broadly applied in image synthesis [17], [18], [19], [20], [21], [22],style transfer [16], [23], [24], [25], [26], [27], image inpainting [28], [29], [30], [31], [32], 3D pose estimation [33], [34], image segmentation [35], [36], [37], image composition [38], [39], [40], image/video colorization [41], [42], [43], [44], [45], [46], image super-resolution [47], [48], domain adaptation [49], [50], [51], [52]. We will analyze and discuss these related applications in details in following section.

Mainly explore methods by focusing on different issues. We will introduce methods with monomodal output in the following four categories: translation that uses cyclic consistency constraints, translation that exceeds cyclic consistency constraints, translation of fine-grained objects, and translation that combines knowledge in other domains. **Translation using a Cycle-consistency Constraint** In the beginning, researchers tried to find new frameworks or constraints to create I2I mappings without labels or pairs. Based on this motivation, a cyclic consistency constraint is proposed, and it has been proven to be an effective strategy to overcome the lack of supervised pairing. **Translation Beyond Cycle-consistency Constraint** However, although the cycle consistency constraint can eliminate the dependence on supervised paired data, it often forces the model to generate a translated image containing all the information of the input image and reconstruct the input image. When two domains require a lot of confusion and heterogeneity instead of small and simple changes in low-level shape and context, the method of using loop loss is usually unsuccessful. Therefore, many UI2I methods focus on translation beyond the cyclic consistency constraint to solve the isomorphism limitation and the large shape deformation problem between the source domain and the target domain. **fine-grained object translation** Most UI2I models that use or exceed the cyclic consistency constraints tend to directly synthesize a new domain and translate the global target style, and rarely consider local object or fine-grained instance translation. But in some application scenarios, such as virtual try-on, we may only need to change a partial object, such as changing pants to skirts, and the other parts remain unchanged. In this case, when the translation involves large shape changes of the instance or multiple different objects, serious frustration will occur. Therefore, research on application examples or object information in UI2I is a growing trend. **Combine knowledge in other fields for translation** In addition, some UI2I attempts to improve network efficiency or translation performance by combining knowledge in other research fields.

III. Methods

This is a dataset comprised of satellite images of New York and their corresponding Google maps pages. The image translation problem involves converting satellite photos to Google maps format, or the reverse, Google maps images to Satellite photos. The dataset is provided on the pix2pix website and can be downloaded as a 255-megabyte zip file. The train folder contains 1,097 images, whereas the validation dataset

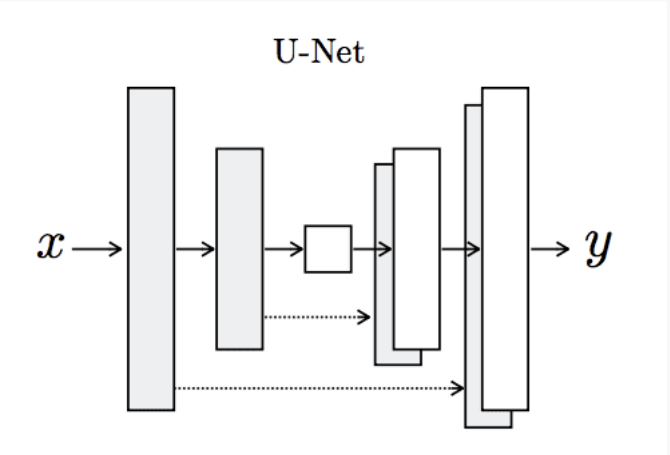


Fig. 2: Architecture of the U-Net Generator Model Taken from Image-to-Image Translation With Conditional Adversarial Networks.

contains 1,099 images. Images have a digit filename and are in JPEG format. Each image is 1,200 pixels wide and 600 pixels tall and contains both the satellite image on the left and the Google maps image on the right. We can prepare this dataset for training a Pix2Pix GAN model in Keras. We will just work with the images in the training dataset. Each image will be loaded, rescaled, and split into the satellite and Google map elements. The result will be 1,097 color image pairs with the width and height of 256×256 pixels. The load images function below implements this. It enumerates the list of images in a given directory, loads each with the target size of 256×512 pixels, splits each image into satellite and map elements and returns an array of each. We can call this function with the path to the training dataset. Once loaded, we can save the prepared arrays to a new file in compressed format for later use. A plot of three image pairs is also created showing the satellite images on the top and Google map images on the bottom. We can see that satellite images are quite complex and that although the Google map images are much simpler, they have color codings for things like major roads, water, and parks. Now that we have prepared the dataset for image translation, we can develop our Pix2Pix GAN model.

A. Developing and Training a Pix2Pix Model

In this section, we will develop the Pix2Pix model for translating satellite photos to Google maps images. The same model architecture and configuration described in the paper was used across a range of image translation tasks. This architecture is both described in the body of the paper, with additional detail in the appendix of the paper, and a fully working implementation provided as open source with the Torch deep learning framework. The implementation in this section will use the Keras deep learning framework based directly on the model described in the paper and implemented in the author's code base, designed to take and generate color images with the size 256×256 pixels.

The architecture is comprised of two models: the discriminator and the generator. The discriminator is a deep convolutional neural network that performs image classification. Specifically, conditional-image classification. It takes both the source image (e.g. satellite photo) and the target image (e.g. Google maps image) as input and predicts the likelihood of whether target image is real or a fake translation of the source image. The discriminator design is based on the effective receptive field of the model, which defines the relationship between one output of the model to the number of pixels in the input image. This is called a PatchGAN model and is carefully designed so that each output prediction of the model maps to a 70×70 square or patch of the input image. The benefit of this approach is that the same model can be applied to input images of different sizes, e.g. larger or smaller than 256×256 pixels. The output of the model depends on the size of the input image but may be one value or a square activation map of values. Each value is a probability for the likelihood that a patch in the input image is real. These values can be averaged to give an overall likelihood or classification score if needed. The define discriminator() function below implements the 70×70 PatchGAN discriminator model as per the design of the model in the paper. The model takes two input images that are concatenated together and predicts a patch output of predictions. The model is optimized using binary cross entropy, and a weighting is used so that updates to the model have half (0.5) the usual effect. The authors of Pix2Pix recommend this weighting of model updates to slow down changes to the discriminator, relative to the generator model during training. The generator model is more complex than the discriminator model. The generator is an encoder-decoder model using a U-Net architecture. The model takes a source image (e.g. satellite photo) and generates a target image (e.g. Google maps image). It does this by first downsampling or encoding the input image down to a bottleneck layer, then upsampling or decoding the bottleneck representation to the size of the output image. The U-Net architecture means that skip-connections are added between the encoding layers and the corresponding decoding layers, forming a U-shape. The image below makes the skip-connections clear, showing how the first layer of the encoder is connected to the last layer of the decoder, and so on.

The encoder and decoder of the generator are comprised of standardized blocks of convolutional, batch normalization, dropout, and activation layers. This standardization means that we can develop helper functions to create each block of layers and call it repeatedly to build-up the encoder and decoder parts of the model. The define generator() function below implements the U-Net encoder-decoder generator model. It uses the define encoder block() helper function to create blocks of layers for the encoder and the decoder block() function to create blocks of layers for the decoder. The tanh activation function is used in the output layer, meaning that pixel values in the generated image will be in the range [-1,1]. The discriminator model is trained directly on real and generated images, whereas the generator model is not. Instead, the generator model is trained via the discriminator model. It is updated to minimize the loss predicted by the discriminator for generated images marked as "real." As such, it is encouraged to generate more real images. The generator is also updated

to minimize the L1 loss or mean absolute error between the generated image and the target image. The generator is updated via a weighted sum of both the adversarial loss and the L1 loss, where the authors of the model recommend a weighting of 100 to 1 in favor of the L1 loss. This is to encourage the generator strongly toward generating plausible translations of the input image, and not just plausible images in the target domain. This can be achieved by defining a new logical model comprised of the weights in the existing standalone generator and discriminator model. This logical or composite model involves stacking the generator on top of the discriminator. A source image is provided as input to the generator and to the discriminator, although the output of the generator is connected to the discriminator as the corresponding "target" image. The discriminator then predicts the likelihood that the generator was a real translation of the source image. The discriminator is updated in a standalone manner, so the weights are reused in this composite model but are marked as not trainable. The composite model is updated with two targets, one indicating that the generated images were real (cross entropy loss), forcing large weight updates in the generator toward generating more realistic images, and the executed real translation of the image, which is compared against the output of the generator model (L1 loss). The define gan() function below implements this, taking the already-defined generator and discriminator models as arguments and using the Keras functional API to connect them together into a composite model. Both loss functions are specified for the two outputs of the model and the weights used for each are specified in the loss weights argument to the compile() function. Typically, GAN models do not converge; instead, an equilibrium is found between the generator and discriminator models. As such, we cannot easily judge when training should stop. Therefore, we can save the model and use it to generate sample image-to-image translations periodically during training, such as every 10 training epochs. We can then review the generated images at the end of training and use the image quality to choose a final model.

The summarize performance() function implements this, taking the generator model at a point during training and using it to generate a number, in this case three, of translations of randomly selected images in the dataset. The source, generated image, and expected target are then plotted as three rows of images and the plot saved to file. Additionally, the model is saved to an H5 formatted file that makes it easier to load later.

Both the image and model filenames include the training iteration number, allowing us to easily tell them apart at the end of training.

Finally, we can train the generator and discriminator models.

The train() function below implements this, taking the defined generator, discriminator, composite model, and loaded dataset as input. The number of epochs is set at 100 to keep training times down, although 200 was used in the paper. A batch size of 1 is used as is recommended in the paper.

Training involves a fixed number of training iterations. There are 1,097 images in the training dataset. One epoch is one iteration through this number of examples, with a batch size of one means 1,097 training steps. The generator is saved



Fig. 3: Plot of Satellite to Google Map Translated Images Using Pix2Pix After 10 Training Epochs

and evaluated every 10 epochs or every 10,970 training steps, and the model will run for 100 epochs, or a total of 109,700 training steps.

Each training step involves first selecting a batch of real examples, then using the generator to generate a batch of matching fake samples using the real source images. The discriminator is then updated with the batch of real images and then fake images.

Next, the generator model is updated providing the real source images as input and providing class labels of 1 (real) and the real target images as the expected outputs of the model required for calculating loss. The generator has two loss scores as well as the weighted sum score returned from the call to train on batch(). We are only interested in the weighted sum score (the first value returned) as it is used to update the model weights.

Finally, the loss for each update is reported to the console each training iteration and model performance is evaluated every 10 training epochs.

REFERENCES

[1] Y. LeCun, S. Chopra, R. Hadsell, M. Ranzato, and F. Huang, "A tutorial on energy-based learning," *Predicting structured data*, vol. 1, no. 0, 2006.

[2] J. Xu, H. Li, and S. Zhou, "An overview of deep generative models," *IETE Technical Review*, vol. 32, no. 2, pp. 131–139, 2015.

[3] A. Oussidi and A. Elhassouny, "Deep generative models: Survey," in *2018 International Conference on Intelligent Systems and Computer Vision (ISCV)*. IEEE, 2018, pp. 1–8.

[4] H.-M. Chu, C.-K. Yeh, and Y.-C. Frank Wang, "Deep generative models for weakly-supervised multi-label classification," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018, pp. 400–415.

[5] R. A. Yeh, C. Chen, T. Yian Lim, A. G. Schwing, M. Hasegawa-Johnson, and M. N. Do, "Semantic image inpainting with deep generative models," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 5485–5493.

[6] M. Tschannen, E. Agustsson, and M. Lucic, "Deep generative models for distribution-preserving lossy compression," in *Advances in Neural Information Processing Systems*, 2018, pp. 5929–5940.

[7] G. E. Hinton and R. R. Salakhutdinov, "Reducing the dimensionality of data with neural networks," *science*, vol. 313, no. 5786, pp. 504–507, 2006.

[8] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," *arXiv preprint arXiv:1312.6114*, 2013.

[9] A. B. L. Larsen, S. K. Sønderby, H. Larochelle, and O. Winther, "Autoencoding beyond pixels using a learned similarity metric," in *International conference on machine learning*. PMLR, 2016, pp. 1558–1566.

[10] X. Chen, Y. Duan, R. Houthooft, J. Schulman, I. Sutskever, and P. Abbeel, "Infogan: Interpretable representation learning by information maximizing generative adversarial nets," in *Advances in neural information processing systems*, 2016, pp. 2172–2180.

[11] J. Donahue, P. Krähenbühl, and T. Darrell, "Adversarial feature learning," *arXiv preprint arXiv:1605.09782*, 2016.

[12] V. Dumoulin, I. Belghazi, B. Poole, A. Lamb, M. Arjovsky, O. Mastropietro, and A. Courville, "Adversarially learned inference," *arXiv preprint arXiv:1606.00704*, 2016.

[13] T. Karras, T. Aila, S. Laine, and J. Lehtinen, "Progressive growing of gans for improved quality, stability, and variation," *arXiv preprint arXiv:1710.10196*, 2017.

[14] Y. Taigman, A. Polyak, and L. Wolf, "Unsupervised cross-domain image generation," *arXiv preprint arXiv:1611.02200*, 2016.

[15] Z. Yi, H. Zhang, P. Tan, and M. Gong, "Dualgan: Unsupervised dual learning for image-to-image translation," in *Proceedings of the IEEE international conference on computer vision*, 2017, pp. 2849–2857.

[16] J.-Y. Zhu, T. Park, P. Isola, and A. A. Efros, "Unpaired image-to-image translation using cycle-consistent adversarial networks," in *Proceedings of the IEEE international conference on computer vision*, 2017, pp. 2223–2232.

[17] T. Park, M.-Y. Liu, T.-C. Wang, and J.-Y. Zhu, "Semantic image synthesis with spatially-adaptive normalization," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2019.

[18] P. Zhu, R. Abdal, Y. Qin, and P. Wonka, "Sean: Image synthesis with semantic region-adaptive normalization," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2020.

[19] C.-H. Lee, Z. Liu, L. Wu, and P. Luo, "Maskgan: Towards diverse and interactive facial image manipulation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 5549–5558.

[20] H. Tang, D. Xu, Y. Yan, P. H. Torr, and N. Sebe, "Local class-specific and global image-level generative adversarial networks for semantic-guided scene generation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2020.

[21] F. Zhan, S. Lu, and C. Xue, "Verisimilar image synthesis for accurate detection and recognition of texts in scenes," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018, pp. 249–266.

[22] F. Zhan, Y. Yu, K. Cui, G. Zhang, S. Lu, J. Pan, C. Zhang, F. Ma, X. Xie, and C. Miao, "Unbalanced feature transport for exemplar-based image translation," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2021.

[23] T. Kim, M. Cha, H. Kim, J. K. Lee, and J. Kim, "Learning to discover cross-domain relations with generative adversarial networks," *arXiv preprint arXiv:1703.05192*, 2017.

[24] Z. Yi, H. Zhang, P. Tan, and M. Gong, "Dualgan: Unsupervised dual learning for image-to-image translation," in *Proceedings of the IEEE international conference on computer vision*, 2017, pp. 2849–2857.

[25] Y. A. Mejjati, C. Richardt, J. Tompkin, D. Cosker, and K. I. Kim, "Unsupervised attention-guided image-to-image translation," in *Advances in Neural Information Processing Systems*, 2018, pp. 3693–3703.

[26] F. Zhan and C. Zhang, "Spatial-aware gan for unsupervised person re-identification," *Proceedings of the International Conference on Pattern Recognition*, 2020.

[27] M. Tomei, M. Cornia, L. Baraldi, and R. Cucchiara, "Art2real: Unfolding the reality of artworks via semantically-aware image-to-image translation," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2019, pp. 5849–5859.

[28] D. Pathak, P. Krahenbuhl, J. Donahue, T. Darrell, and A. A. Efros, "Context encoders: Feature learning by inpainting," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2016.

[29] J.-Y. Zhu, R. Zhang, D. Pathak, T. Darrell, A. A. Efros, O. Wang, and E. Shechtman, "Toward multimodal image-to-image translation," in *Advances in Neural Information Processing Systems*, 2017, pp. 465–476.

[30] Y. Song, C. Yang, Z. Lin, X. Liu, Q. Huang, H. Li, and C.-C. J. Kuo, "Contextual-based image inpainting: Infer, match, and translate," in *Proceedings of the European Conference on Computer Vision (ECCV)*, September 2018.

[31] M.-Y. Liu, X. Huang, A. Mallya, T. Karras, T. Aila, J. Lehtinen, and J. Kautz, "Few-shot unsupervised image-to-image translation," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, October 2019.

[32] L. Zhao, Q. Mo, S. Lin, Z. Wang, Z. Zuo, H. Chen, W. Xing, and D. Lu, "Uctgan: Diverse image inpainting based on unsupervised cross-space translation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2020.

[33] H.-Y. Fish Tung, A. W. Harley, W. Seto, and K. Fragkiadaki, "Adversarial inverse graphics networks: Learning 2d-to-3d lifting and image-to-image translation from unpaired supervision," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, Oct 2017.

[34] S. Li, S. Gunel, M. Ostrek, P. Ramdya, P. Fua, and H. Rhodin, "Deformation-aware unpaired image translation for pose estimation on laboratory animals," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2020.

[35] Q. Yang, N. Li, Z. Zhao, X. Fan, E. I.-C. Chang, and Y. Xu, "Mri cross-modality neuroimage-to-neuroimage translation," 2018.

[36] X. Guo, Z. Wang, Q. Yang, W. Lv, X. Liu, Q. Wu, and J. Huang, "Gan-based virtual-to-real image translation for urban scene semantic segmentation," *Neurocomputing*, vol. 394, pp. 127–135, 2020.

[37] R. Li, W. Cao, Q. Jiao, S. Wu, and H.-S. Wong, "Simplified unsupervised image translation for semantic segmentation adaptation," *Pattern Recognition*, vol. 105, p. 107343, 2020.

[38] F. Zhan, S. Lu, C. Zhang, F. Ma, and X. Xie, "Adversarial image composition with auxiliary illumination," in *Proceedings of the Asian Conference on Computer Vision*, 2020.

[39] F. Zhan, H. Zhu, and S. Lu, "Spatial fusion gan for image synthesis," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2019, pp. 3653–3662.

[40] ——, "Scene text synthesis for efficient and effective deep network training," *arXiv preprint arXiv:1901.09193*, 2019.

[41] R. Zhang, J.-Y. Zhu, P. Isola, X. Geng, A. S. Lin, T. Yu, and A. A. Efros, "Real-time user-guided image colorization with learned deep priors," *arXiv preprint arXiv:1705.02999*, 2017.

[42] P. L. Suárez, A. D. Sappa, and B. X. Vintimilla, "Infrared image colorization based on a triplet dcgan architecture," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops*, 2017, pp. 18–23.

[43] M. He, D. Chen, J. Liao, P. V. Sander, and L. Yuan, "Deep exemplar-based colorization," *ACM Transactions on Graphics (TOG)*, vol. 37, no. 4, pp. 1–16, 2018.

[44] B. Zhang, M. He, J. Liao, P. V. Sander, L. Yuan, A. Bermak, and D. Chen, "Deep exemplar-based video colorization," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2019, pp. 8052–8061.

[45] Z. Xu, T. Wang, F. Fang, Y. Sheng, and G. Zhang, "Stylization-based architecture for fast deep exemplar colorization," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 9363–9372.

[46] J. Lee, E. Kim, Y. Lee, D. Kim, J. Chang, and J. Choo, "Reference-based sketch image colorization using augmented-self reference and dense semantic correspondence," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2020.

[47] Y. Yuan, S. Liu, J. Zhang, Y. Zhang, C. Dong, and L. Lin, "Unsupervised image super-resolution using cycle-in-cycle generative adversarial networks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, June 2018.

[48] Y. Zhang, S. Liu, C. Dong, X. Zhang, and Y. Yuan, "Multiple cycle-in-cycle generative adversarial networks for unsupervised image super-resolution," *IEEE transactions on Image Processing*, vol. 29, pp. 1101–1112, 2019.

[49] Z. Murez, S. Kolouri, D. Kriegman, R. Ramamoorthi, and K. Kim, "Image to image translation for domain adaptation," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2018.

[50] J. Cao, O. Katzir, P. Jiang, D. Lischinski, D. Cohen-Or, C. Tu, and Y. Li, "Dida: Disentangled synthesis for domain adaptation," 2018.

[51] A. H. Liu, Y.-C. Liu, Y.-Y. Yeh, and Y.-C. F. Wang, "A unified feature disentangler for multi-domain image translation and manipulation," in *Advances in neural information processing systems*, 2018, pp. 2590–2599.

[52] F. Zhan, C. Xue, and S. Lu, "Ga-dan: Geometry-aware domain adaptation network for scene text detection and recognition," in *Proceedings of the IEEE International Conference on Computer Vision*, 2019, pp. 9105–9115.